

Regenta

App modular para Android y web — arquitectura, módulos, dependencias y planes

La idea central

No se construye la app dos veces. Se escribe la lógica una sola vez, en un framework capaz de compilar hacia distintos destinos (targets): un instalable para Android y un paquete de HTML/JS/CSS para el navegador. La modularidad (inventario, ventas, facturación) se resuelve aparte, organizando el proyecto en paquetes independientes.

1. Un solo framework, varios destinos de compilación

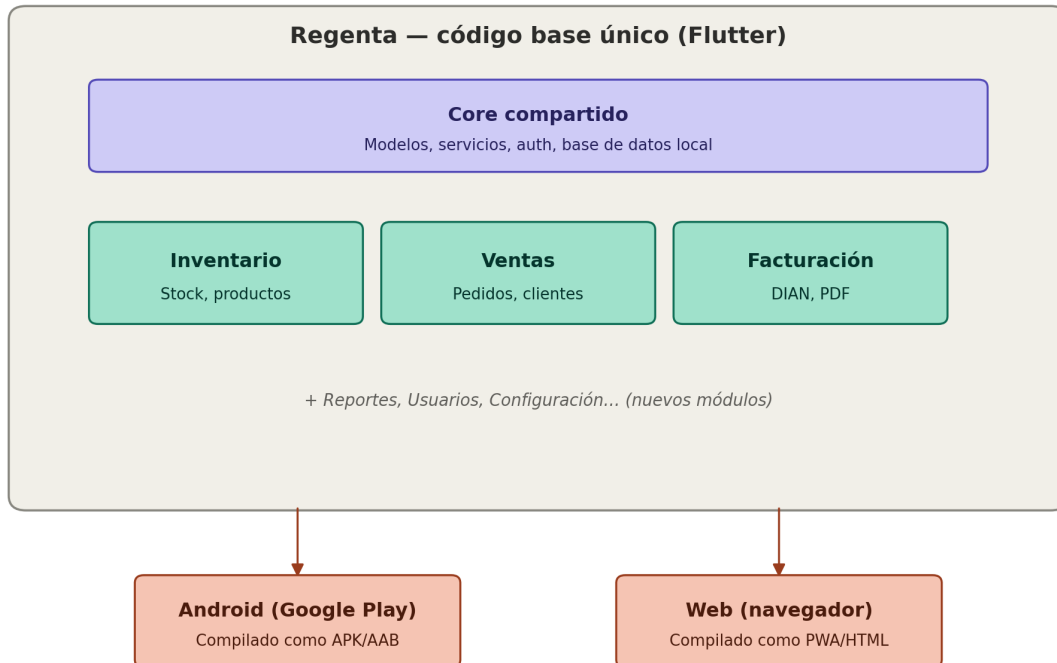
Con Flutter (Dart) el mismo código se traduce en un APK/AAB nativo para Android y, con `flutter build web`, en HTML/JS/CSS para el navegador. No se mantienen dos bases de código distintas para cada plataforma.

2. La modularidad va aparte, por paquetes

Se organiza el proyecto como un monorepo (por ejemplo con la herramienta `me1os`) con paquetes independientes:

```
mi_app/  
packages/  
core/ -> modelos, auth, cliente HTTP, DB local (Drift)  
inventario/ -> modulo de stock y productos  
ventas/ -> modulo de pedidos y clientes  
facturacion/ -> modulo de DIAN, PDF, impuestos  
apps/  
app_movil/ -> importa los modulos que necesite  
(web usa el mismo app_movil, solo cambia  
el target de build)
```

Diagrama de arquitectura



Código base único en Flutter: un núcleo compartido, los módulos de negocio, y dos targets de compilación.

Cada módulo expone su propia interfaz (pantallas y rutas) y su propia lógica de negocio, y se comunica con los demás únicamente a través del paquete core compartido (eventos, repositorios). Esto permite activar o desactivar módulos por cliente sin tocar los demás.

3. Diferencias reales entre Android y web a prever

- **Persistencia local:** Drift (SQLite) funciona en Android; en web usa IndexedDB por debajo. El mismo código funciona, pero el rendimiento y los límites de almacenamiento cambian.
- **Facturación electrónica:** Si aplica normativa como la de la DIAN en Colombia, la generación de PDF y la firma digital conviene resolverlas en un backend, no en el cliente.
- **Diseño responsive:** Se usa LayoutBuilder / MediaQuery en Flutter para que la misma pantalla se adapte bien tanto a un celular como a un navegador ancho.

Alternativa

Si se prefiere separar completamente el frontend web del app: Angular para la web + Capacitor para empaquetarlo como app Android. Implica más trabajo de sincronización entre dos bases de interfaz, pero da más control nativo por plataforma si en algún momento se necesitan funciones específicas de Android que Flutter no cubra bien.

Módulos adicionales sugeridos

Pensando en un sistema de gestión típico para un negocio (retail, distribución, etc.), estos son los módulos que se agregarían, agrupados por prioridad:

Para completar el ciclo del negocio

- **Compras / Proveedores:** Sin esto, inventario se queda "vacío" cuando ya no hay stock. Registra órdenes de compra, proveedores y costos.
- **Clientes / CRM básico:** Historial de compras por cliente, crédito, datos de contacto. Ventas y facturación lo necesitan tarde o temprano.
- **Caja / Punto de venta (POS):** Distinto de "Ventas" porque maneja apertura/cierre de caja, métodos de pago y cambio, útil si hay venta presencial.

Para dar valor sin ser core

- **Reportes y dashboards:** Ventas por período, productos más vendidos, rotación de inventario. Es lo que hace que el dueño del negocio realmente use la app a diario.
- **Alertas de inventario:** Stock mínimo, productos por vencer, reposición automática sugerida.
- **Multi-sucursal / multi-bodega:** Si el negocio crece a más de un punto físico, cambia el modelo de datos de inventario, conviene pensarlo desde el diseño aunque no se implemente de una.

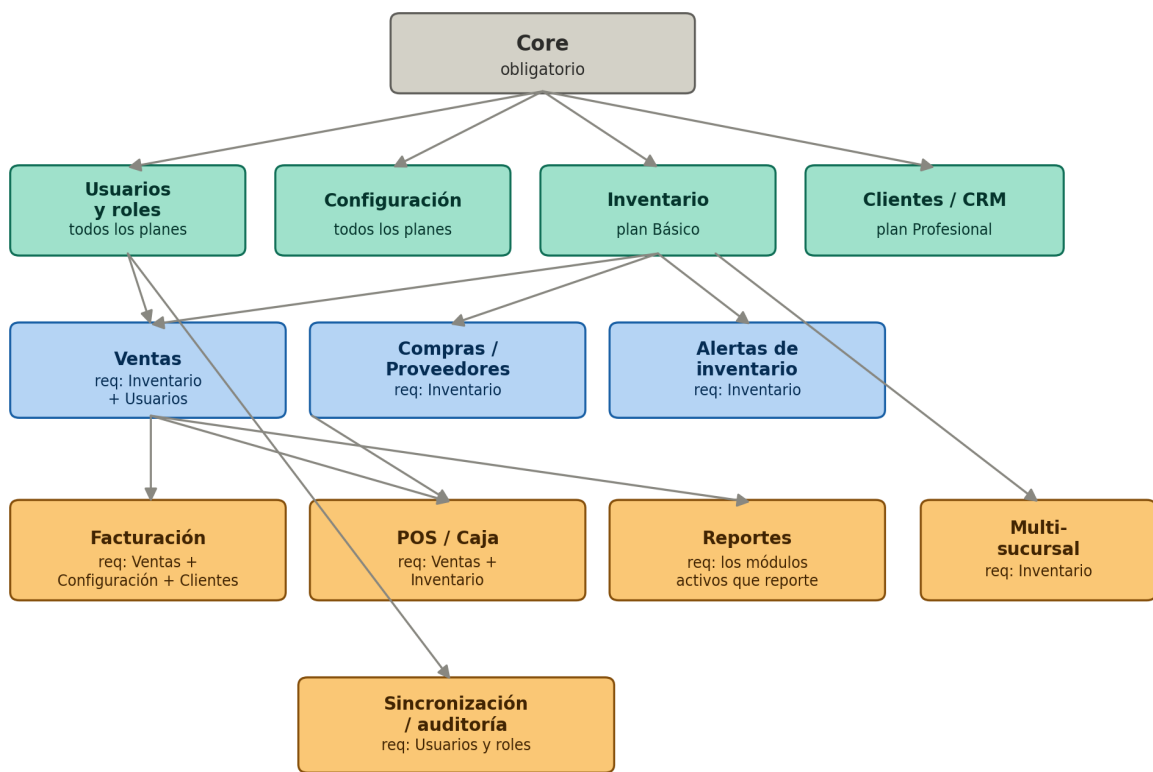
Estructurales (no son "features" pero sí módulos)

- **Usuarios y roles / permisos:** Quién puede ver ventas, editar precios, o solo consultar inventario. Casi siempre se subestima al inicio y es doloroso agregarlo después.
- **Configuración / parámetros del negocio:** Datos fiscales, moneda, impuestos, logo para PDFs de factura.
- **Sincronización / auditoría:** Si varias personas usan la app a la vez (celular + web), hay que saber quién cambió qué y cuándo, y resolver conflictos de datos.

Orden de implementación sugerido después de Inventario, Ventas y Facturación: **Usuarios/roles** → **Compras/Proveedores** → **Reportes** → **Clientes/CRM** → **POS** → **Multi-sucursal**, porque los primeros tres hacen que los módulos actuales funcionen de forma completa y segura; los últimos son crecimiento.

Dependencias entre módulos

Este mapa muestra de qué módulo depende cada uno, para poder activar o desactivar funcionalidades según el plan contratado (Básico, Profesional, Empresarial) sin romper nada.



Verde = incluido desde el plan Básico. Azul = requiere módulos de nivel 1. Ámbar = plan Profesional/Empresarial.

Tabla de referencia

Módulo	Depende de	Plan sugerido
Core	—	Obligatorio, todos los planes
Usuarios y roles	Core	Todos los planes
Configuración del negocio	Core	Todos los planes
Inventario	Core	Básico
Clientes / CRM	Core	Profesional
Ventas	Core, Inventario, Usuarios y roles	Básico
Compras / Proveedores	Core, Inventario	Profesional

Módulo	Depende de	Plan sugerido
Alertas de inventario	Core, Inventario	Profesional
Facturación	Core, Ventas, Configuración, Clientes	Profesional
POS / Caja	Core, Ventas, Inventario	Empresarial / add-on
Reportes y dashboards	Core + los módulos activos que reporte	Profesional
Multi-sucursal	Core, Inventario (afecta Ventas/Compras)	Empresarial
Sincronización / auditoría	Core, Usuarios y roles	Empresarial

Resumen: Core es obligatorio para todos los planes. El plan Básico habilita Usuarios, Configuración, Inventario y Ventas. El plan Profesional agrega Clientes, Compras, Alertas, Facturación y Reportes. El plan Empresarial suma POS, Multi-sucursal y Sincronización/auditoría.

Planes de Regenta

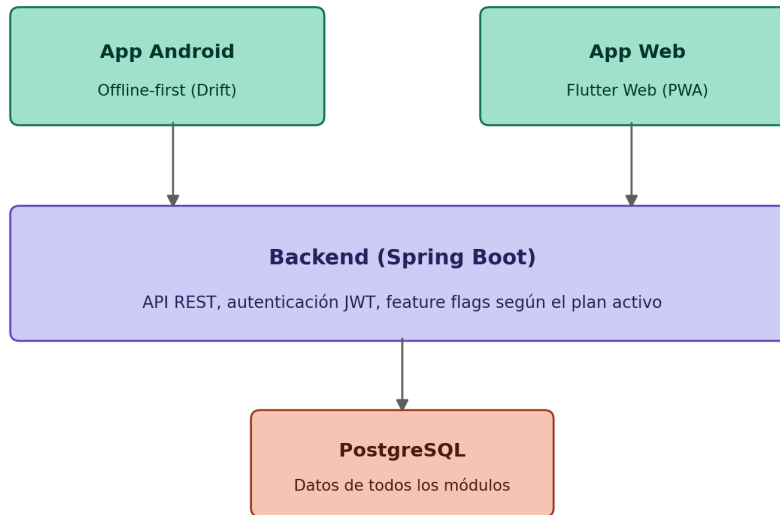
Los planes son acumulativos: cada uno incluye todos los módulos del plan anterior, respetando las dependencias definidas arriba (por ejemplo, nunca se activa Facturación sin que Ventas ya esté incluida).

Plan Básico	Plan Profesional	Plan Empresarial
Módulos incluidos	Módulos incluidos	Módulos incluidos
Core, Usuarios y roles, Configuración, Inventario, Ventas	Todo lo del Básico + Clientes/CRM, Compras/Proveedores, Alertas de inventario, Facturación, Reportes	Todo lo del Profesional + POS/Caja, Multi-sucursal, Sincronización/auditoría
Usuarios	Usuarios	Usuarios
1-2 usuarios	Hasta 5-10 usuarios	Usuarios ilimitados
Ideal para	Ideal para	Ideal para
Negocio pequeño que empieza a organizar stock y ventas	Negocio en crecimiento con varios empleados y proveedores	Cadenas o negocios con varias sucursales y cajas
Soporte	Soporte	Soporte
Correo electrónico	Correo + chat	Prioritario

El nombre "Regenta" viene de "regentar" un negocio: llevar el control completo de su operación (inventario, ventas, facturación y todo lo que se active según el plan) desde una sola aplicación, en Android y en la web.

Arquitectura detallada: Android, web y backend

Backend recomendado: **Spring Boot**. Aprovecha los 10 años de experiencia en Java/Spring, y convierte este proyecto en un portafolio directamente alineado con procesos de selección técnica como el de GFT Technologies.



Dos clientes Flutter (Android y web) consumen el mismo backend Spring Boot, que persiste en PostgreSQL.

Lado Android (Flutter → target móvil)

- **State management:** Riverpod, ya usado en la app de bookmarks.
- **Persistencia local:** Drift (SQLite), offline-first: el vendedor registra una venta sin señal y se sincroniza después.
- **Navegación:** go_router, con rutas protegidas por rol usando el módulo de Usuarios/roles.
- **Escaneo de códigos:** mobile_scanner para leer códigos de barras en Inventario/POS, ventaja real frente a la web donde el acceso a cámara es más limitado.
- **Notificaciones:** flutter_local_notifications + Firebase Cloud Messaging para alertas de stock bajo.
- **Almacenamiento seguro:** flutter_secure_storage para el token JWT, usando el keystore de Android.
- **Sincronización en segundo plano:** workmanager para subir ventas pendientes cuando vuelve la conexión.

Lado web (mismo código, target flutter build web)

- **Código y lógica:** El mismo core y lógica de negocio del móvil; solo cambian los widgets que dependen del tamaño de pantalla (LayoutBuilder / MediaQuery).
- **Persistencia:** Drift usa sq_lite3.wasm por debajo en vez de SQLite nativo: mismo código Dart, solo cambia el motor.

- **PWA:** Flutter genera manifest.json y service worker automáticamente, por lo que la web queda instalable desde el navegador.
- **Cámara / scanner:** Más limitaciones de navegador; conviene tener siempre una alternativa de captura manual del código de producto.
- **Deploy:** Cloudflare Pages, igual que en CivixOps: build estático, rápido y gratis en el tier inicial.

Backend (Spring Boot)

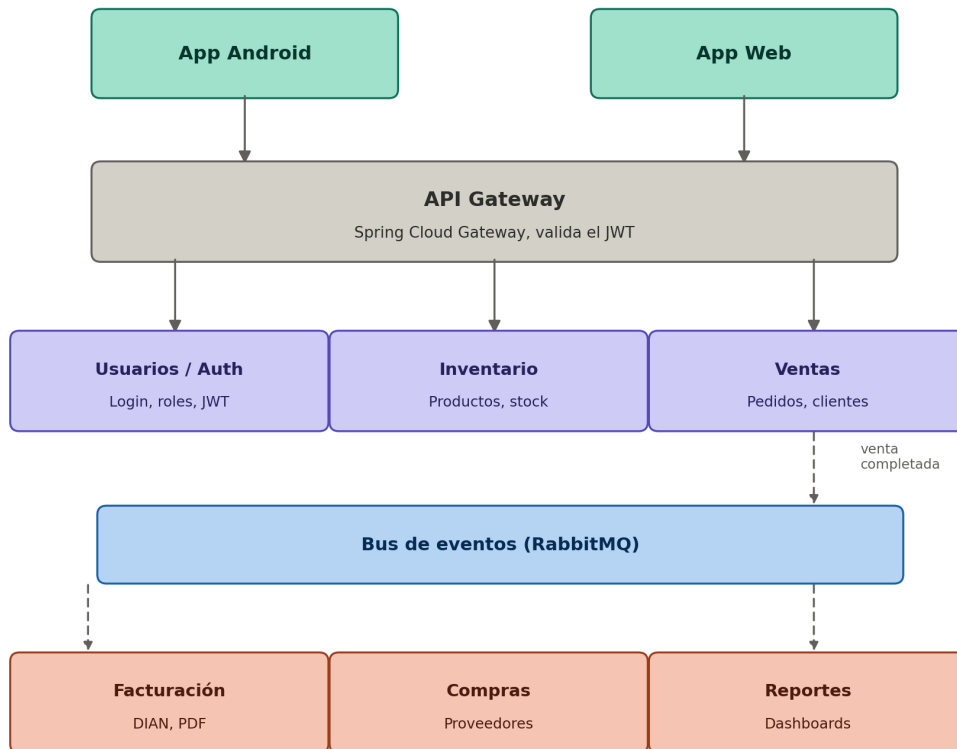
Estructurado como **monolito modular** al inicio, no microservicios: con un equipo pequeño, los microservicios añaden complejidad operativa innecesaria en esta etapa.

```
regenta-backend/
core/ -> auth, usuarios, config del tenant/negocio
inventario/ -> productos, stock
ventas/ -> pedidos
facturacion/ -> DIAN, PDF
compras/ -> proveedores
...
```

- **Base de datos:** PostgreSQL: más robusto que MySQL para los reportes y agregaciones que se necesitarán.
- **Autenticación:** Spring Security + JWT, con el rol y el plan del tenant embebidos en el token o resueltos en cada request.
- **Feature flags por plan:** Una anotación/interceptor tipo @ModuloRequerido("FACTURACION") que verifica si el plan del negocio incluye ese módulo antes de dejar pasar la petición. El control de qué módulos están activos vive en el backend, no solo en el cliente: si alguien manipula la app, el backend igual bloquea.
- **Documentación:** springdoc-openapi para generar Swagger automáticamente, útil tanto para que Claude Code genere el cliente Dart como para portafolio.
- **Deploy:** Railway + PostgreSQL, igual que en CivixOps.

Actualización: backend con microservicios

Decisión tomada: en lugar del monolito modular descrito arriba, el backend de Regenta se construye como microservicios. Cada módulo se vuelve un servicio independiente y desplegable por separado, lo que encaja bien con el requisito de activar módulos según el plan: simplemente no se despliega o no se enruta el servicio que el plan no incluye. Los dos clientes (Android y web) no cambian nada de su lado, ambos siguen hablando solo con el API Gateway, así que no hay lógica duplicada por plataforma.



API Gateway como único punto de entrada; comunicación síncrona con Usuarios, Inventario y Ventas; comunicación asíncrona vía bus de eventos hacia Facturación, Compras y Reportes.

Piezas clave del diseño

- **API Gateway:** Spring Cloud Gateway como único punto de entrada tanto para Android como para web. Valida el JWT una vez ahí y enruta a cada microservicio, sin duplicar lógica de auth en cada cliente.
- **Comunicación síncrona (REST):** Entre el Gateway y servicios como Usuarios, Inventario y Ventas, porque el usuario necesita respuesta inmediata (por ejemplo, verificar si hay stock).
- **Comunicación asíncrona (eventos):** Para todo lo que no bloquea al usuario: cuando Ventas termina un pedido, publica un evento `venta_completada` en el bus (RabbitMQ, más sencillo de operar que Kafka a esta escala); Facturación lo consume para generar el PDF/DIAN, y Reportes lo consume para actualizar sus métricas, sin que Ventas sepa que existen.
- **Base de datos por servicio:** Cada microservicio tiene su propio esquema o instancia de PostgreSQL, lo que separa los módulos a nivel de datos y da el aislamiento necesario para activar o desactivar por plan.

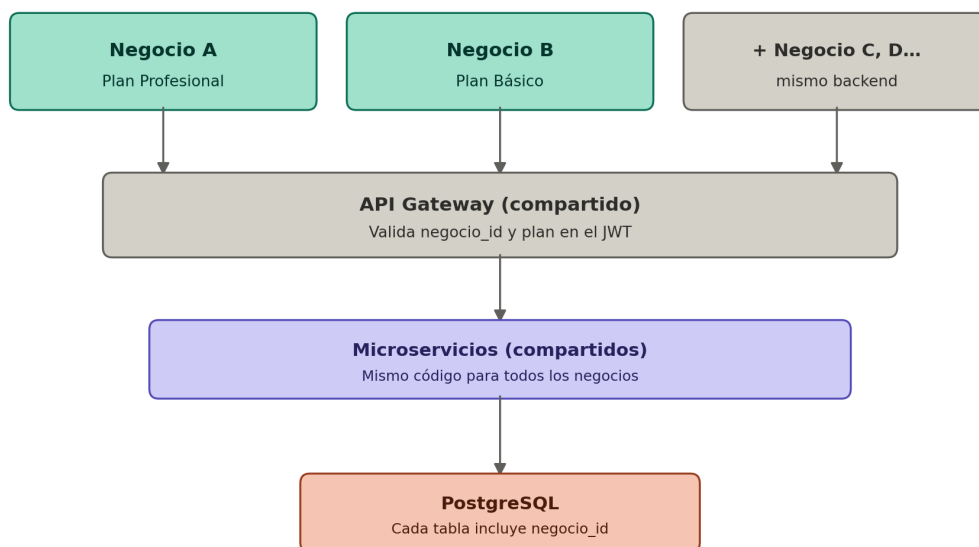
Complejidad adicional a asumir conscientemente

- **Service discovery:** Eureka, o el DNS interno si se usa Docker/Kubernetes, para que el gateway sepa dónde está cada servicio.
- **Transacciones distribuidas:** Si Ventas descuenta stock e Inventario falla, ya no hay un `@Transactional` que cubra ambos; se maneja con el patrón Saga o compensación.

- **Observabilidad:** Logs centralizados y trazabilidad (por ejemplo Spring Cloud Sleuth + Zipkin), porque un solo request ahora cruza varios servicios.
- **Despliegue:** Cada microservicio es un contenedor Docker separado. Railway soporta múltiples servicios dentro de un mismo proyecto, así que no es necesario saltar directo a Kubernetes.

Multi-tenancy: cómo se atiende a cada cliente nuevo

Cuando se vende Regenta a un negocio nuevo, ese negocio NO recibe infraestructura propia. Un solo backend desplegado sirve a todos los negocios clientes (modelo multi-tenant); los datos se separan de forma lógica, no física.



Varios negocios comparten el mismo Gateway, los mismos microservicios y la misma base de datos; cada tabla incluye una columna `negocio_id`.

Cómo funciona en la práctica

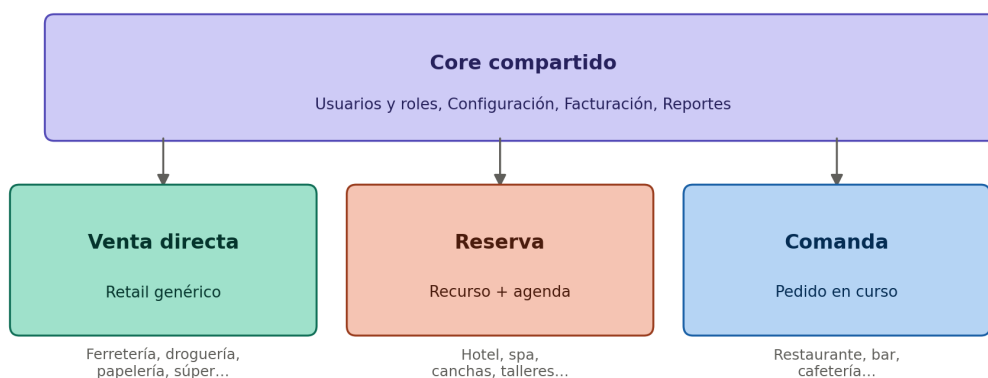
- **Una sola tabla, muchos negocios:** Cada tabla del backend (productos, ventas, facturas, etc.) tiene una columna `negocio_id`. Toda query queda filtrada por ese id, así que nunca se mezclan datos entre negocios aunque vivan en la misma base de datos física.
- **El JWT es la llave:** El token de cada usuario lleva el claim `negocio_id` (además del plan y el rol). El Gateway y cada microservicio usan ese claim para filtrar automáticamente: el usuario no puede pedir datos de otro negocio aunque quisiera, porque el filtro vive en el backend, no en el cliente.
- **Vender a un cliente nuevo = una fila nueva, no un servidor nuevo:** Se crea un registro en la tabla negocios (nombre, plan contratado, datos fiscales) y su primer usuario admin asociado a ese `negocio_id`. No hay que tocar Docker, Railway, ni desplegar nada nuevo.

- **Row-Level Security como respaldo:** Postgres permite definir a nivel de base de datos que ninguna query puede ver filas de un negocio_id distinto al de la sesión, como capa extra de seguridad ante un posible error en el filtro de algún servicio.

¿Cuándo tendría sentido infraestructura dedicada? Solo para un cliente empresarial que exija por contrato o regulación que sus datos estén físicamente aislados; eso sería la excepción, no la regla, y normalmente se cobra más caro para justificar el costo operativo extra.

Generalización: patrones operativos, no un módulo por nombre de negocio

Corrección de diseño importante: ferretería, papelería, droguería, supermercado o tienda de ropa no son negocios distintos a nivel de sistema — todos son el mismo patrón (venden productos con inventario). Crear un módulo por cada nombre de negocio no escala. En lugar de eso, Regenta define un número pequeño de **patrones operativos**, y cada negocio elige uno y lo configura, sin necesidad de código nuevo.



El negocio elige un patrón y configura categorías/atributos — no se crea módulo nuevo

El Core es el mismo para todos los negocios; solo existen tres patrones operativos, cada uno cubre muchos tipos de negocio distintos vía configuración.

Patrón Venta directa: mismo módulo, configuración distinta

Ferretería, papelería y droguería usan exactamente el mismo servicio de Inventario y Ventas. Lo único que cambia entre ellas es la configuración: categorías de producto (las define cada negocio, no vienen fijas en el código) y atributos adicionales según la categoría.

	Ferretería	Papelería	Droguería
Categorías (las crea el negocio)	Herramientas, Tornillería, Pintura	Útiles escolares, Oficina	Medicamentos, Cuidado personal

	Ferretería	Papelería	Droguería
Atributos extra por categoría	Unidad de medida (metro, kilo)	— (producto simple)	Lote, vencimiento, registro sanitario
Regla especial	—	—	Reportar a INVIMA, alertar vencimientos

Para que esto funcione, el modelo de datos de Inventario debe ser flexible desde el diseño:

```
productos (id, negocio_id, nombre, categoria_id, precio, stock,
atributos JSONB) -> campos que varían por categoría
categorias (id, negocio_id, nombre) -> cada negocio crea las suyas
atributos_categoria(categoria_id, nombre_campo, tipo, obligatorio)
-> ej: "Medicamentos" exige lote + vencimiento;
"Tornillería" no exige nada extra
```

Así, cuando aparezca un cliente de vidriería o veterinaria, no se programa nada nuevo: el dueño crea sus categorías y ya está usando el mismo patrón de Venta directa. Los únicos casos que sí necesitan un patrón distinto son los que cambian la *forma* de la transacción, no solo el nombre del producto.

Patrón Reserva (ej. Hotel) y patrón Comanda (ej. Restaurante)

Estos dos patrones sí cambian la forma de la transacción: Reserva no vende un producto, reserva un recurso por un bloque de tiempo; Comanda mantiene un pedido abierto que acumula ítems antes de cerrarse. Un negocio de hotelería usa el patrón Reserva; un restaurante usa el patrón Comanda:

Patrón	Módulo	Depende de	Equivale a (Venta directa)
Reserva	Recursos (habitaciones, canchas...)	Core	Inventario
Reserva	Reservas	Core, Recursos, Usuarios	Ventas
Reserva	Check-in / Check-out	Core, Reservas	Extensión de Ventas
Comanda	Menú	Core	Inventario
Comanda	Mesas	Core	Sin equivalente directo
Comanda	Comandas / Pedidos	Core, Menú, Mesas, Usuarios	Ventas
Comanda	Cocina (KDS)	Core, Comandas	Alertas (cola de trabajo)

Usuarios y roles: siempre por negocio

El módulo de Usuarios y roles es del Core (no de un patrón), pero cada negocio administra su propio conjunto de usuarios y roles, aislados por negocio_id igual que el resto de los datos. Roles como Administrador y Gerente son comunes a cualquier negocio; cada patrón sugiere plantillas de rol adicionales sobre el mismo mecanismo genérico de permisos: Vendedor/Cajero en Venta directa, Recepcionista en Reserva, Mesero y Cocina en Comanda. El motor de roles y permisos no cambia entre patrones ni entre negocios, solo las plantillas que cada uno activa.